SQL DDL Script Writing Exercises (With Answers)

Exercise 1 — Creating a Database and Table

Create a database named:

`InventoryDB`

Write the statement so that the database is created only if it does not already exist.

Make `InventoryDB` the default database.

Then create a table named `Product` with the following columns. Do not create the tables if it already exists.

Column	Data Type	Constraint
ProductID	Number	Primary Key
ProductName	String	Cannot be NULL
Price	Money	6 total digits, 2 digits after decimal point
Quantity	Number	

Then create a table named `Category` with the following columns:

Column	Data Type	Constraint
CategoryID	Number	Primary Key
CategoryName	String	Must be Unique

Answer:

```
CREATE DATABASE IF NOT EXISTS InventoryDB;
USE InventoryDB;
CREATE TABLE IF NOT EXISTS Product(
ProductID INT PRIMARY KEY,
ProductName VARCHAR(100) NOT NULL,
Price DECIMAL(6, 2),
Quantity INT
);
CREATE TABLE Category(
CategoryID INT PRIMARY KEY,
CategoryName VARCHAR(50) UNIQUE
);
```

Exercise 2 — Relate the Tables using a Foreign Key

Create a foreign key so that:

```
Product.CategoryID
```

references:

```
Category.CategoryID
```

Make sure the tables are created in the correct order. This follows the same foreign-key concept demonstrated in the presentation without reusing the classroom entities.

Answer:

```
CREATE DATABASE IF NOT EXISTS InventoryDB;
USE InventoryDB;
CREATE TABLE Category(
CategoryID INT PRIMARY KEY,
CategoryName VARCHAR(50) UNIQUE
);

CREATE TABLE IF NOT EXISTS Product(
ProductID INT PRIMARY KEY,
ProductName VARCHAR(100) NOT NULL,
Price DECIMAL(6, 2),
Quantity INT,
CategoryID INT,
FOREIGN KEY (CategoryID) REFERENCES Category(CategoryID)
);
```

Exercise 3 — Using `ALTER TABLE` to Add Columns

Assume the following table already exists (copy/paste this in your script):

```
CREATE TABLE Customer (
    CustomerID INT PRIMARY KEY,
    CustomerName VARCHAR(100) NOT NULL
);
```

Add the following columns to the Customer table (do not recreate the table):

```
Email       String of length 100
Phone       String of length 20
City        String of length 30
```

Answer:

```
ALTER TABLE Customer
ADD Email VARCHAR(100),
ADD Phone VARCHAR(20),
ADD City VARCHAR(30);
```

Exercise 4 — Modifying, Renaming, and Dropping Columns

Assume the following table already exists (copy/paste this in your script):

```
CREATE TABLE Employee (
    EmployeeID INT PRIMARY KEY,
    EmployeeName VARCHAR(50) NOT NULL,
    JobTitle VARCHAR(30),
    Extension INT
);
```

Using **MySQL**, write statements to:

1. Change `EmployeeName` to length of 100.
2. Rename `JobTitle` to `Position`.
3. Remove the `Extension` column.

Answer:

```
ALTER TABLE Employee
MODIFY COLUMN EmployeeName VARCHAR(100) NOT NULL,
RENAME COLUMN JobTitle TO Position,
DROP COLUMN Extension;
```

Exercise 5 — Adding a Constraint with `ALTER TABLE`

Assume this MySQL table already exists (copy/paste this in your script):

```
CREATE TABLE Customer (
    CustomerID INT PRIMARY KEY,
    CustomerName VARCHAR(75) NOT NULL,
    Email VARCHAR(100)
);
```

Write an `ALTER TABLE` statement that makes `Email` unique.

Name the constraint:

`uq_customer_email`

Answer:

```
ALTER TABLE Customer
ADD CONSTRAINT uq_customer_email UNIQUE(Email);
```

Exercise 6 — Using `DROP`

Suppose the following two tables exist (copy/paste this in your script):

```
CREATE TABLE Category (
    CategoryID INT PRIMARY KEY,
    CategoryName VARCHAR(50) UNIQUE
);

CREATE TABLE Product (
    ProductID INT PRIMARY KEY,
    ProductName VARCHAR(50) NOT NULL,
    Quantity INT,
    CategoryID INT,
    FOREIGN KEY (CategoryID)
        REFERENCES Category(CategoryID)
);
```

`Product` contains a foreign key referencing `Category`.

Write the SQL statements needed to remove both tables.

Answer:

```
DROP TABLE Product;
DROP TABLE Category;
```
